Game Studio Zhovten Games
https://zhovten-games.github.io/

Published under the Creative Commons Attribution-ShareAlike 4.0 license.

Literate Programming: Donald Knuth, WEB, and Contemporary Workflows

Sam Starling
Zhovten Games, sam@phantom-draft.com
0009-0009-2621-6372
Oksana Dubinetska
Zhovten Games, oksana.dubinetska@phantom-draft.com
0009-0003-8777-8412

Abstract: Donald Knuth's 1984 article Literate Programming proposed writing programs primarily for human readers and only secondarily for machine execution. This review reconstructs that argument through WEB and traces its relevance to source generation, executable documents, reproducible environments, and LLM-assisted development.

Keywords: literate programming, Donald Knuth, WEB, software engineering, reproducible research, executable documents, LLM-assisted development, Prompt-Literate Workflow, literate design

Methodological Framework of the Review

The breadth of the issues addressed is not accidental. Donald Knuth's article is not narrowly focused documentation for the WEB system, but a methodological text written by a practising programmer who speaks simultaneously as an engineer and as a theorist of the profession. In discussing a specific tool, Knuth addresses the structure of a program, the order in which it should be explained, style, portability, development costs, the publication of code, and the future form of programming environments. Any contemporary commentary on this work therefore inevitably extends beyond the reconstruction of WEB and requires several engineering lineages to be compared.

At the same time, the tools and practices discussed in this review are not treated as a single genealogical tree in which every modern mechanism literally descends from WEB. For the purposes of the subsequent analysis, it is important to distinguish three types of relationship. The first is a direct or historically proximate continuation of WEB: above all, CWEB and noweb. The second is a practice with an analogous instrumental logic: for example, Org Babel tangling, which employs weaving/tangling mechanisms, although this alone does not establish a direct genealogical connection. The third is a conceptual association, or a productive comparison: WEB ↔ notebooks, WEB ↔ workflows based on large language models (LLMs)[1], WEB ↔ integrated development environment (IDE)[2] navigation and context maps. In the latter case, the issue is the recurrence of a similar problem: how to connect code, explanation, reading order, navigation, and a verifiable result.

This comparison follows the logic that Donald Knuth himself sets out in the Related Work section: the strength of WEB lay not in the invention of each individual element, but in bringing existing ideas together into an integrated working method.

Table of Contents

Main Article

[1] A large language model, or LLM, is a statistical model trained to continue and transform text according to context and instructions.

[2] An integrated development environment, or IDE, combines editing, navigation, execution, debugging, and other development tools.

- **C–F. Example / source / tangle / weave** — the prime-number example, the original WEB file, the machine-oriented TANGLE projection, and the human-oriented WEAVE projection.
- **G. Additional Bells and Whistles** — WEB's additional capabilities and the transition from an instructional example to production-level complexity.
- **H. Occam's Razor** — the limits of the tool and the need for methodological restraint.
- **I. Portability** — the portability of the literate approach beyond the original TeX / Pascal pairing.
- **J. Programs as Webs** — the program as a network of named fragments, dependencies, and explanatory relationships.
- **K. Stylistic Issues** — style, section naming, and the readability of literate source.
- **L. Economic Issues** — the costs of introducing, maintaining, and consistently applying literate programming.
- **M. Related Work** — related lineages: structured programming, documentation systems, and executable-document practices.
- **N. Retrospect and Prospects** — a retrospective assessment of the method and its future prospects.
- **Conclusions 1–5** — which aspects of Knuth's idea persist literally, which have shifted towards reproducible research and notebook environments, and which are re-emerging in LLM-assisted workflows.

Repository Navigation

Canonical Source of the Article

- public repository root: `https://github.com/Zhovten-Games/literate-programming`;
- canonical source of the review: `01-LiterateProgramming-KnuthWEBandContemporaryWorkflows/01-LiterateProgramming-KnuthWEBandContemporaryWorkflows.md`;
- author metadata file: `01-LiterateProgramming-KnuthWEBandContemporaryWorkflows/authors.yml`;
- bibliography inventory layer: `01-LiterateProgramming-KnuthWEBandContemporaryWorkflows/Bibliography/Original.md`;
- sectioned bibliography JSON: `01-LiterateProgramming-KnuthWEBandContemporaryWorkflows/Bibliography/*.json`.

Companion Suite

- root of the companion suite: `02-Literate-Companion-Implementations/`;
- shared README for the companion suite: `02-Literate-Companion-Implementations/README.md`;
- Makefile for verification runs: `02-Literate-Companion-Implementations/Makefile`;
- English branch of the examples: `02-Literate-Companion-Implementations/examples/en/`;
- Russian branch of the examples: `02-Literate-Companion-Implementations/examples/ru/`.

Deterministic Companion Examples 01–06

- `01-cweb` — the more canonical CWEB route, with an explicit two-branch `tangle/weave` model;
- `02-noweb-like` — the primary lightweight C++ remaster, centred on a `tangle-first workflow`;
- `03-org-babel` — a plain-text literate workflow in an Org-mode / Emacs environment, with tangling;
- `04-quarto` — an executable-document route for publication-ready HTML / computational publishing;
- `05-jupyter` — a notebook route with particular attention to execution, hidden state, and reproducible execution;
- `06-rmarkdown` — a reproducible report workflow based on R Markdown / knitr / Pandoc.

Prompt-Literate Workflow and the Boundary LLM Example

- public repository for the Prompt-Literate Workflow methodology: `https://github.com/IRONCREED/prompt-literate-workflow`;
- role: a reusable methodology repository and GitHub template repository;
- integration in `literate-programming`: a root submodule/scaffold mounted at `prompt-literate-workflow/`;
- methodology path used by the companion examples: `02-Literate-Companion-Implementations/methodology/prompt-literate-workflow/`;
- article-specific local specialisation for Knuth's prime-number problem: `02-Literate-Companion-Implementations/methodology/extensions/primes-example/`;

- `07-prompt-literate` — a demonstrative application of Prompt-Literate Workflow (PLW) to the prime-number example, rather than a definition of the method itself.

Working Files within `07-prompt-literate`

- `COMPANION.md` — entry point for the companion example;
- `primes.plan.md` — human-authored plan and canonical source for the prompt-literate example;
- `CONTRACTS.md` — chunk contracts for the prime-number task;
- `SCENARIOS.md` — validation scenarios and acceptance criteria;
- `prompts/fill-chunks.prompt.md` — a bounded prompt for filling only the permitted `LLM-TODO` chunks;
- `prompts/review-generated-code.prompt.md` — a prompt for review, rather than for rewriting the architecture;
- `generated/.gitkeep` — an empty directory for a future accepted generated artifact;
- `tests/smoke-check.sh` — verification of the accepted generated code;
- `output.expected.txt` — expected output markers;
- `TRACE.md` — a log of the model, prompt run, review, manual edits, validation, and acceptance decision.

Main Article

Opening Remarks

The historical significance of Donald Knuth's article *Literate Programming* becomes particularly clear in retrospect. The text was published in 1984, before the later canonisation of pattern language in *Design Patterns* (1994) (Gamma et al. 1994) and long before *Clean Architecture* (2017) (Martin 2017). In this sense, it represents one of the early attempts to impose methodological order on the complexity of software development not only through the structure of code, but also through the structure of its explanation.

Literate Programming can be read as a precise formulation of a problem that remains relevant: a program must be not only executable but also explainable. Knuth proposes changing whom programming is primarily addressed to: rather than treating the instruction of a computer as the primary task, a programmer should first explain to a human reader

what the computer is intended to do (Knuth 1984). This shift does not negate machine execution, but subordinates it to the broader task of organising a program in an order suited to human understanding.

In 2026, this formulation has acquired additional relevance because the software-development workflow itself has changed. If the classical model can be described as a movement from one human being to another and then to a machine, an AI-assisted workflow increasingly involves a chain in which a human formulates an intention for a machine, while one computational system helps produce input for another. A prompt, however, is not in itself a literate source. A contemporary AI-based approach can be regarded as a substantive continuation of literate programming only if it preserves an engineering discipline: explanation → specification → code → tests → artifact. Without this chain, what accelerates is not so much the understanding of a program as the production of code with insufficient verifiability and a weak explanatory structure.

In this review, WEB is treated as the original form of this idea. In Donald Knuth's model, a single source generates two projections: a document for the human reader and a program for the machine. This relationship between explanation, code, and the build mechanism connecting them serves as the principal criterion for the analysis that follows. The purpose of the review is not to reproduce the Pascal example as a self-contained historical object. The example's codebase has been remastered in C++, while the primary demonstration route has been implemented in a noweb-like form: it preserves named chunks and tangling without requiring entry into the more elaborate CWEB/TeX toolchain.

The subsequent analysis compares several instrumental and methodological lineages:

- **CWEB** (Knuth and Levy, n.d.),
- **noweb-like C++** (Ramsey, n.d.),
- **Org Babel** ("Org Manual: Extracting Source Code," n.d.),
- **Quarto / Jupyter / R Markdown** ("Quarto Documentation," n.d.; "Project Jupyter Documentation," n.d.; "R Markdown Documentation," n.d.).

The technical branches are documented in companion materials within the code repository. The main text follows Donald Knuth's article and tests its central proposition: whether a program can first be organised in an order suited to human understanding and only then transformed into machine-executable code.

The conclusion returns to several questions: how literate programming relates to the later language of engineering patterns; which of Knuth's claims were realised literally; which ideas shifted into the domain of reproducible research and notebook environments; and what is now re-emerging around LLMs. Particular attention will be given to the proposition

that an LLM should receive not an unconstrained request, but a bounded operation over a human-authored literate plan, chunk contracts, bounded prompts, tests/smoke-check, and TRACE.

There is no full-fledged `07-llm` companion here that would be equivalent in reproducibility to CWEB, noweb, Org Babel, Quarto, Jupyter, or R Markdown: LLM generation depends on the model, version, context, and execution mode. Instead, a boundary example, `07-prompt-literate`, has been added to demonstrate the application of Prompt-Literate Workflow. Its status and limitations will be examined in detail in the conclusion.

Prompt-Literate Workflow was initially formulated within the present review and subsequently refined through its practical application to an independent core component of a game project. This pilot demonstrated the need to separate the method's compact, universal core from a project-local extension layer. Domain-specific rules must not silently override the basic invariants: they should be formalised as local extensions, while the inclusion of a local rule in the core requires a separate process of generalisation and promotion into the methodology's foundational layer.

A. Introduction

Donald Knuth begins by establishing a professional norm. Structured programming has already made programs more reliable and easier to understand, but this is not sufficient. He sees the next step not in the introduction of a new control construct, but in changing the way a program is presented.

The image of the programmer as an essayist is not merely decorative. Understanding must be embedded in the structure of the program itself: concepts are introduced in an order suited to the reader, while formal and informal means of exposition reinforce one another.

B. The WEB System

In the second section, Donald Knuth demonstrates that *literate programming* is not a metaphor, but a concrete system: WEB. Its purpose is to combine a document language with a programming language so that a single source can generate two distinct results: a readable description for a human reader and an executable program for a machine.

Donald Knuth proposes that a complex program should be understood as a network of simple parts and the relationships between them.

It is useful here to distinguish three terms:

- **WEB** — from the English word *web* (a network or interwoven structure). In Donald Knuth's model, a program is a network of named sections and relationships between them, rather than merely a linear file.
- **TANGLE** — from the verb *to tangle* (to intertwine or rearrange into a machine-oriented order). The utility assembles the compiler-oriented source code of the program.
- **WEAVE** — from the verb *to weave* (to interlace or compose into a readable document). The utility produces a human-oriented document describing the program.

Technically, WEB is bilingual, and its underlying idea is designed to be portable beyond a specific technology stack. In the original version, TeX serves as the document-formatting language and Pascal as the programming language, but Donald Knuth explicitly emphasises that the principle is not tied to this pairing: TeX could be replaced with Scribe or Troff, while Pascal could be replaced with C, LISP, FORTRAN, ALGOL, assembly language, or other languages.

The central scheme of the section is **WEAVE / TANGLE**. WEAVE produces a document that describes the program and facilitates its maintenance; TANGLE produces a machine-executable program. Both results are generated from the same source, so the code and its documentation do not diverge into two independent versions of the truth and do not require manual synchronisation.

This scheme also establishes the criterion for evaluating the contemporary branches identified in the opening remarks: the primary question is not which tools they use, but whether they preserve the relationship between explanation, code, and a verifiable artifact.

C–F. The Prime-Number Example: Source, Tangle, and Weave

This section follows the methodological choice stated earlier: the primary remaster is implemented in noweb-like C++. This format was selected not because of any limitations in CWEB, but as a lightweight adaptation of Donald Knuth's idea to a contemporary team workflow. Even the basic discipline of `canonical source -> tangling -> generated artifact -> smoke-check` already imposes a non-trivial entry threshold; making a separate readable-document generation branch mandatory would bring the example closer to classical WEB/CWEB, but would make it less suitable for demonstrating the minimal practical core of the method.

A more canonical contrast has been preserved in `02-Literate-Companion-Implementations/examples/en/01-cweb/`: this is a direct C-family lineage from WEB to CWEB

(Knuth and Levy, n.d.), with a two-branch model: `primes.w -> ctangle -> primes.c -> cc/gcc -> primes -> output.txt` and `primes.w -> cweave -> primes.tex -> pdftex -> primes.pdf`. This demonstrates that the complete tangle/weave model remains operational. For the main review, however, `02-noweb-like` was selected in order to demonstrate the portability of the principle without requiring entry into the CWEB/TeX toolchain.

Historically, Donald Knuth's example follows the discussion of structured programming: the question is no longer only how to structure control flow, but also how to structure the explanation of a program (Dijkstra 1972: 26–39).

The C–F subsections below therefore do not reproduce the entire codebase. Instead, they identify the four projections of Donald Knuth's example and immediately compare them with the corresponding stages of our remaster pipeline. The complete noweb-like source is presented once, in the "Remaster" subsection.

C. Donald Knuth's Readable Example

Section C presents the readable projection of the example, which is also discussed later in Section F: the reader is given not a linear Pascal file, but a composition of the program as an explanation.

```
The program prints the first 1000 prime numbers:
  1. define the table parameters;
  2. introduce the table of discovered prime numbers;
  3. generate the prime numbers;
  4. print the table page by page;
  5. provide the reader with a table of contents, sections, and
an index of names.
```

D. The Original WEB File

Section D presents the original `PRIMES.WEB`, that is, the material written directly by the author: the documentation layer + Pascal + WEB commands.

In our remaster equivalent, `primes.nw` serves as the source of truth.

noweb chunk markers and syntax:

- `<<program>>=` defines a named chunk called `program`.
- `<<name>>` refers to another named chunk.

- `@` terminates the current chunk.
- `notangle -Rprogram` selects `program` as the root chunk.
- `notangle` expands the chunk references and generates ordinary C++.
- The C++ compiler never sees the noweb markers; it sees only the generated `primes.cpp`.

E. The Machine-Oriented Projection: TANGLE

Section E establishes the machine-oriented projection: `TANGLE` transforms `PRIMES.WEB` into `PRIMES.PAS`.

In our remaster, the corresponding artifact is `primes.cpp`.

```
primes.nw -> notangle -Rprogram -> primes.cpp -> g++ -> primes ->
output.txt
```

`output.txt` serves as a verification artifact (smoke-check), rather than as a readable document.

F. The Human-Oriented Projection: WEAVE

Section F defines a separate branch for generating a readable document: in WEB/CWEB, this is `WEAVE/cweave -> TeX -> PDF`.

Remaster: Why the noweb-like Version Is Deliberately Lightweight

`02-noweb-like` presents a tangle-first C++ remaster. It preserves the minimal practical core of the method: a canonical literate source, named chunks, a generated machine artifact, and a smoke-check verification artifact. A separate branch for generating a readable document is not mandatory in this version.

The material below is presented as a unified literate form of the remaster: the explanation remains adjacent to the corresponding chunks rather than being extracted into separate, repetitive sections.

The top level defines the program's intent and order of composition:

```
# Printing primes: a C++ reboot

This program prints the first 1000 prime numbers.
```

```
The top-level structure is deliberately simple:
generate the prime table, then print it.

<<program>>=
#include <cstddef>
#include <iomanip>
#include <iostream>
#include <vector>

<<constants>>
<<types>>
<<declarations>>

int main() {
    const auto primes = generate_primes(PRIME_COUNT);
    print_table(primes);
    return 0;
}

<<prime-generation>>
<<table-output>>
@
```

The output parameters define the table layout and preserve the connection with Donald Knuth's original example:

```
The output format follows Knuth's original example:
50 rows per page, 4 columns, 10 character positions per column.

<<constants>>=
constexpr std::size_t PRIME_COUNT = 1000;
constexpr std::size_t ROWS_PER_PAGE = 50;
constexpr std::size_t COLUMNS_PER_PAGE = 4;
constexpr int COLUMN_WIDTH = 10;
@
```

The container for the table of prime numbers is assigned a distinct semantic role:

```
The table of primes is represented as a growing sequence.
The alias makes the role of the container explicit.

<<types>>=
using PrimeTable = std::vector<int>;
@
```

The function declarations establish a separation of responsibilities between generation, candidate validation, and output:

```
Generation, candidate checking, and output are introduced
as separate operations.

<<declarations>>=
PrimeTable generate_primes(std::size_t count);
bool is_prime_candidate(int candidate, const PrimeTable& primes);
void print_table(const PrimeTable& primes);
@
```

Prime-number generation preserves the algorithm of the remaster version without altering its logic:

```
<<prime-generation>>=
PrimeTable generate_primes(std::size_t count) {
    PrimeTable primes;
    if (count == 0) return primes;
    primes.push_back(2);

    for (int candidate = 3; primes.size() < count; candidate +=
2) {
        if (is_prime_candidate(candidate, primes)) {
            primes.push_back(candidate);
        }
    }
    return primes;
}
```

```
bool is_prime_candidate(int candidate, const PrimeTable& primes)
{
    for (int p : primes) {
        if (p > candidate / p) return true;
        if (candidate % p == 0) return false;
    }
    return true;
}
@
```

The output phase preserves the page-by-page layout and the calculation of indices by row and column:

```
<<table-output>>=
void print_table(const PrimeTable& primes) {
    std::size_t page_number = 1;
    std::size_t page_offset = 0;

    while (page_offset < primes.size()) {
        std::cout << "The First " << primes.size()
                  << " Prime Numbers --- Page " << page_number <<
"\n\n";

        for (std::size_t row = 0; row < ROWS_PER_PAGE; ++row) {
            for (std::size_t column = 0; column <
COLUMNS_PER_PAGE; ++column) {
                const std::size_t index = page_offset + row +
column * ROWS_PER_PAGE;
                if (index < primes.size()) {
                    std::cout << std::setw(COLUMN_WIDTH) <<
primes[index];
                }
            }
            std::cout << '\n';
        }

        std::cout << '\n';
        ++page_number;
```

```
        page_offset += ROWS_PER_PAGE * COLUMNS_PER_PAGE;
    }
}
@
```

`primes.nw` remains the canonical literate source; `primes.cpp` serves as the generated machine artifact; and `output.txt` serves as the smoke-check output.

G. Additional Bells and Whistles

In Section G, Donald Knuth makes a transition: the prime-number example is useful as a demonstration, but it does not establish WEB's suitability for real-world development. A system must be capable of handling not only an instructional fragment, but also large programs, different compilers, non-standard conventions, typography, portability, and the accumulation of technical exceptions. Otherwise, it is not a method, but merely an elegant toy constructed for an article.

Donald Knuth lists several capabilities of WEB83 that were not required in `PRIMES.WEB`, but become necessary in larger programs: manual control over WEAVE formatting, the ability to pass fragments through TANGLE almost verbatim, control over spacing and line beginnings, octal and hexadecimal constants, ASCII-based character encoding, a string pool with a checksum, numeric macros, and simple compile-time arithmetic.

The significance of this section does not lie in the features themselves. Many of them now appear to be compensations for the limitations of Pascal and older compilers. In C++, some of these problems simply disappear: the language provides proper strings, `constexpr`, local variables, standard containers, formatters, and build systems.

The point is that a literate tool must be capable of working with the messy realities of a language and platform, rather than only with an idealised example. The prime-number example explains the principle, not the entire production toolchain.

For the noweb-like C++ remaster, this can be expressed as follows:

```
Toy example:
  <<program>>
  <<constants>>
  <<types>>
  <<prime-generation>>
  <<table-output>>
```

```
Real-world program:
  <<headers>>
  <<platform-specific-code>>
  <<configuration>>
  <<generated-bindings>>
  <<tests>>
  <<diagnostics>>
  <<build-notes>>
```

Thus, the “additional bells and whistles” are not embellishments in the ordinary sense. They form a layer that allows the literate source to remain the principal source even when the project ceases to be clean and small. The circumstances in which such discipline is genuinely justified will be examined in the conclusion of this review; Donald Knuth himself also proceeds to qualify the universality of WEB and does not present it as a tool for every possible case.

H. Occam’s Razor

Section H serves as a methodological qualification of the preceding section. After a lengthy list of capabilities, it is easy to draw the false conclusion that a literate system should continue expanding indefinitely and incorporate everything: formatting, portability, macro expansion, publication, build processes, diagnostics, and environment management. Donald Knuth instead invokes Occam’s razor: complexity should be introduced only where it is genuinely required by the task, rather than merely because the tool is technically capable of accommodating yet another feature.

This is particularly important in considering the history of WEB, whose original strength lay in connecting the explanation and the code of a program within a single working process. Yet the same system historically accumulated ceremonial overhead and a substantial entry threshold: numerous commands, a typographic layer, and a dependence on the conventions of a particular school of development. Later branches therefore embodied a different response: not the indefinite expansion of the core, but its reduction to a minimally viable discipline.

In this sense, noweb, as used in our remaster, is significant as an engineering reduction rather than as an "anti-WEB". It preserves the essential relationship between named fragments and tangling, while removing much of the ceremonial overhead. This neither supersedes classical WEB nor makes noweb a universal solution; it demonstrates that a literate approach requires proportion. If a system attempts to encompass every task simultaneously, it begins to succumb to its own complexity before it can systematically improve understanding.

I. Portability

In Section I, Donald Knuth moves from clarity of exposition to portability. TeX and its associated software had to operate across numerous machines and operating systems. WEB proved useful precisely because it made it possible to preserve a shared program source while still accounting for platform-specific differences.

Donald Knuth first describes a bootstrapping scheme: given `TANGLE.PAS`, `TANGLE.WEB`, and `WEAVE.WEB`, one can obtain a working version of TANGLE, use it to generate `WEAVE.PAS`, and then build the entire WEB system and programs such as TeX.

Donald Knuth then qualifies the practical limitations of the model. Real machines differ in character sets, file conventions, terminals, data packing, floating-point arithmetic, and compiler quality. It is therefore methodologically unsound to expect a single `TANGLE.PAS` or `TANGLE.WEB` to operate everywhere without modification. System-dependent changes are inevitable.

WEB's solution is **change files**. TANGLE and WEAVE read not only the primary WEB file, but also a separate `.CH` file containing substitutions for a particular system. As a result, the master source remains stable, while platform-specific modifications are maintained separately. Donald Knuth emphasises that `TANGLE.WEB` itself remains effectively unchanged: it is `TANGLE.CH` that varies, while the core logic of TANGLE remains shared across systems.

Donald Knuth's approach to literate programming is not detached from build and distribution processes.

For our noweb-like C++ remaster, this can be represented as follows:

```
canonical source:
  primes.nw

generated artifact:
  primes.cpp
```

```
platform overlay:
  primes.linux.patch
  primes.windows.patch
  config/platform.hpp
  CMake presets
```

In other words, a direct contemporary analogue of .CH does not need to be a separate "change file" in the classical WEB format. In modern practice, the range of productive analogies is broader:

```
change file → patch layer
change file → platform adapter
change file → environment-specific config
change file → build preset
change file → overlay
```

The principle, however, remains the same: the primary explanatory source should not be disrupted by every platform-specific requirement. If Windows, Linux, or a particular compiler requires divergent behaviour, that divergence should be isolated within an adaptation layer. Otherwise, the canonical source rapidly becomes a collection of platform-specific exceptions in which the central idea is obscured by adaptation noise.

Contemporary branches implement this principle through different means, ranging from master-source discipline to the management of reproducible environments.

The distinction between classical WEB and the executable-document branch becomes substantive at this point. For Donald Knuth, portability primarily concerns transferring a program between machines. In Quarto/Jupyter/R Markdown, portability often means something different: the ability to rebuild a computational report with the same dependencies, the same data, and the same execution order. This is not a literal `.CH`, but it reflects the same underlying concern: the source must survive a change of environment.

Contemporary development introduces an additional layer: portability is no longer limited to differences between operating systems and compilers. Increasingly, the issue is a

reproducible working environment: WSL[3], containers[4], cloud IDEs[5], CI/CD runners[6], dev containers[7], and managed environments. Under these conditions, the question changes from "will the program build on another machine?" to "can the entire process be restored: source, tooling, dependencies, build commands, verification artifacts, and execution order?"

It is illustrative that the companion examples for this review were reproduced within a WSL environment under Windows 11. This is a contemporary form of portability: Windows serves as the working platform, WSL as the Linux execution environment, and the examples themselves are verified using specific CLI tools[8]: `ctangle`, `notangle`, `emacs`, `quarto`, `jupyter`, `Rscript`, `pandoc`, and `g++`. Portability thus becomes not an abstract property of the source text, but a verifiable state of the entire toolchain.

The changing role of the environment is particularly clear. In classical WEB, system-dependent differences were isolated in `.CH` files; in contemporary practice, an analogous function may be performed by a WSL profile, a container, a lockfile[9], a CI configuration, a build preset[10], or reproducible-run documentation. The principle remains unchanged: the primary explanatory source should not disintegrate because of the peculiarities of a particular machine. What must now be recorded alongside the source is not only the platform-specific modification, but also the means of reconstructing the execution environment itself.

[3] Windows Subsystem for Linux, or WSL, is a Windows feature that runs Linux environments and Linux tools without requiring the separate setup of a virtual machine or a dual-boot configuration.

[4] A container is an isolated runtime environment in which an application runs with explicitly specified dependencies and configuration.

[5] A cloud IDE is a development environment delivered through a browser or remote infrastructure rather than only through the author's local machine.

[6] A CI/CD runner is the executor of an automated pipeline for checking, building, or publishing a project.

[7] A dev container describes a reproducible development environment, including tools, extensions, dependencies, and project commands.

[8] A command-line interface, or CLI, controls a tool through terminal commands, which makes checks and automation easier to reproduce.

[9] A lockfile records exact dependency versions and related metadata so that repeated installation is not affected by incidental upstream updates.

[10] A build preset is a named build profile with predefined parameters such as compiler, mode, and output directory.

J. Programs as Webs

Section J returns to the article's central architectural idea: a program is structured neither as a simple sequence of steps nor as a tidy top-down tree. Donald Knuth proposes that it should instead be understood as a network — a web — in which named fragments are connected by meaning, dependencies, and explanatory order. One node may introduce an idea, another refine its constraints, and a third provide its implementation; the reader moves through this network deliberately, rather than being compelled to follow the order imposed by the compiler.

> *Retrospective: This section appears methodologically farsighted. It is not a direct prediction of contemporary tools, but Donald Knuth's model maps closely onto later problems of navigation within complex codebases: dependency graphs, module maps, symbol navigation in IDEs, call/reference navigation, knowledge graphs, and project context maps; LLM context maps can be mentioned here only as a preliminary horizon. For contemporary practice, the implication is straightforward: understanding a program is increasingly constructed through a network of relationships between fragments, rather than through the linear inspection of files.*

The WEB model cannot be reduced to the attractive metaphor that "everything is connected to everything else". Its value lies in the fact that the network of relationships is recorded in a verifiable source from which both the machine-oriented result and the human-oriented explanation are generated. This is why Section J occupies a position between portability and style in the logic of our review: first, the integrity of the source is preserved across platforms; then, the source is understood as a network; only after that do we consider how this network should be named and presented.

K. Stylistic Issues

In Section K, Donald Knuth moves from the architecture of WEB to the style of working within it. Once a program becomes an explanatory text, style ceases to be cosmetic: it becomes part of the engineering of understanding.

The central question of the section is how sections should be named and organised. Donald Knuth distinguishes between macros and named fragments: small technical abbreviations may remain macros, but larger parts of a program should receive human-readable names. Such a name need not be formally exhaustive; it should express the meaning of the fragment with sufficient precision, without burdening the reader with excessive detail.

A poor section name in the noweb-like C++ remaster:

```
<<code-for-loop>>
```

A better name:

```
<<generate prime candidates>>
```

Better still, if the fragment genuinely expresses an action:

```
<<print one page of the prime table>>
```

Donald Knuth emphasises the balance between formal and informal exposition. A section name should not be turned into a complete specification of every variable and precondition: if it is, it ceases to assist the reader and begins to imitate a legal contract. Yet it must not remain excessively general either. A well-named section should be precise enough for the reader to understand its role before reading the code.

For the C++ version, this means that fragments should be named according to their purpose rather than their syntax:

```
<<constants>>
<<types>>
<<declarations>>
```

— are acceptable for auxiliary blocks, but semantic names are preferable for logic:

```
<<generate the requested number of primes>>
<<test whether a candidate is prime>>
<<print the prime table page by page>>
```

Donald Knuth also discusses control structures separately. If the control flow is non-standard, the section name should make this explicit. The danger lies not in a `goto`, loop, or early exit as such, but in an unexplained control transition. In a contemporary C++ version, the same concern applies to `break`, `continue`, `return`, exceptions, guard clauses, and other means of altering the ordinary flow of a function.

Across all branches, style remains a working instrument: the easier it is to navigate the names and roles of fragments, the more robust the understanding of the program becomes.

Later engineering culture develops a similar discipline through naming conventions: the names of classes, methods, APIs, modules, UI blocks, and BEM-like schemes[11], as codified in style guides and readable-code practices. This is not literate programming in itself, but it reflects the same effort to construct a readable structure: a name should communicate the role of a fragment before the reader examines its implementation.

This section therefore draws a boundary between literate programming and contemporary documentation generators. Javadoc, Doxygen, or comment extraction can produce an API reference, but this is not necessarily literate programming. Such tools more often describe an already existing code structure. In Donald Knuth's approach, style operates prospectively.

L. Economic Issues

In Section L, Donald Knuth asks a pragmatic question: what does WEB cost? He begins by considering direct computational expenses. TANGLE takes approximately as much time as compiling the resulting Pascal file; WEAVE also runs reasonably quickly, although TeX typesetting is already noticeably more expensive. It is therefore not always sensible to rebuild and print the documentation in full several times a day.

Documentation is rarely printed today, but the central economic argument does not concern processor time or the cost of paper. Donald Knuth argues that the total time he spends writing and debugging a WEB program is no greater than the time required to write and debug an ordinary ALGOL or Pascal program, despite the substantially more extensive documentation that results. The additional time devoted to explanation is recovered through reduced debugging effort, because the mode of exposition forces the author to clarify ideas earlier. When a program is written "for oneself", it is easy to rely on shortcuts that later become sources of error. When a program is written as an explanation, it becomes more difficult for the author to deceive himself.

For the noweb-like C++ remaster, this means that the cost of the method must be stated plainly:

```
a higher entry threshold;
the need for a tangle/render workflow;
the need to maintain canonical-source discipline;
the requirement not to edit the generated .cpp manually;
the need to maintain the explanation alongside the code.
```

[11] BEM, or Block–Element–Modifier, is a UI-component naming convention in which a name indicates the block, its element, and a state or modification variant.

The benefits are equally concrete:

```
intent is recorded more clearly;
review becomes easier;
returning to the code later becomes easier;
introducing a new reader becomes easier;
code, tests, and documentation are easier to connect.
```

The economics of the different branches vary in their entry thresholds and risk profiles, but the underlying exchange follows the same model: greater discipline at the outset in return for lower maintenance costs later.

This section also provides a bridge to contemporary reproducibility practices. In research and data science, the literate approach became economically justified because reports, code, graphs, results, and validation began to be assembled from a single source. In widespread practice, this is visible in executable documents and reproducible computation: the idea has become part of the everyday workflow of science and data science.

The section concludes with an irony: Donald Knuth worries that literate programs may appear so complete to their authors that they will feel compelled to publish them everywhere. Once code becomes text, the temptation to aestheticise it inevitably emerges.

For the purposes of this review, the boundary must remain clear: the value lies not in making a program resemble a "work of art", but in making it better explained and more readily verifiable.

M. Related Work

In Section M, Donald Knuth removes the aura of solitary invention from WEB. He states explicitly that the system contains nothing "really new": he assembled ideas that had existed for a long time and combined them into a working form. This is an important act of intellectual candour: WEB is presented as a node in the history of programming, typography, and the publication of algorithms.

The first important source is George Forsythe, for whom a useful algorithm constitutes a contribution to knowledge, while its publication is a form of scholarly work. This directly supports Donald Knuth's argument: a program can be not merely a functional mechanism, but also a publishable intellectual object. The idea of a program as literature follows naturally from this position, but without romanticisation: the issue is the scholarly form in which an algorithm is presented.

Donald Knuth then refers to Pierre-Arnoul de Marneffe and holon programming, followed by Dijkstra, Hoare, Dahl, Wirth, and Naur. The central intellectual lineage becomes visible here: abstraction, structured programming, a balance between formal and informal description, and the publication of well-written programs. WEB does not supersede this tradition; it attempts to give it an instrumental form.

Particularly important is the episode involving Tony Hoare, who encouraged Donald Knuth to publish TeX. For Knuth, this posed a challenge: it is one thing to publish polished toy problems, and quite another to expose a substantial real-world program with all its compromises, portability concerns, and untidy details. This problem generated the practical need for WEB: to make a large program accessible to public reading.

The section shows that literate programming emerged at the intersection of several lineages:

```
structured programming;
the publication of algorithms;
the typography of programs;
documentation as part of design;
a balance between formal and informal exposition;
the attempt to make a large program readable.
```

The contemporary landscape of related practices has a similar structure. CWEB, noweb, Org Babel, R Markdown, Quarto, and Jupyter do not form a single direct lineage. They are better understood as a family of practices in which different domains address related engineering problems in different ways. This landscape can be read in terms of a two-layer influence: direct or historically proximate continuations of WEB — above all, CWEB and noweb — and the more widespread layer of executable documents and reproducible computing, which addresses related problems through different means.

The conclusion should therefore avoid searching for a single “true successor”. CWEB is closest genealogically; noweb, in the simplicity of its mechanism; Org Babel, in its plain-text practice; and Quarto/Jupyter/R Markdown, in their institutional scale. Each implements part of the same engineering task in its own way. What matters is that direct continuation of WEB, instrumental affinity, and productive conceptual association should not be conflated.

Ultimately, WEB is a synthesis rather than a solitary invention. This, in turn, legitimises the comparative approach: contemporary branches should likewise be read not as copies of WEB, but as different ways of continuing its central inquiry — how to make a program readable and verifiable.

N. Retrospect and Prospects

In the final section, Donald Knuth himself tempers the rhetoric of his manifesto. He acknowledges that the creators of new languages are generally inclined to overestimate the significance of their own experience, and that his work with WEB is too deeply shaped by his personal preferences.

The article ends not with a declaration of victory, but with a cautious test: will the method work beyond its author and his environment?

At the same time, Donald Knuth formulates a strong conclusion: in his experience, WEB makes it possible to create programs that are more portable, more comprehensible, and easier to maintain, while the method works not only for small examples but also for large systems. For him, WEB is the result of a reversal in the relationship between typography and computing: the computer was first applied to typesetting, and typography then returned to the centre of computer science as a means of making programs readable.

Donald Knuth then explicitly limits the intended audience. WEB was not designed for everyone. Its user must be prepared to work with several languages simultaneously: WEB, TeX, Pascal, and the algorithmic errors of the program itself. It is not a low-friction tool, but an environment for people who enjoy writing and explaining what they do. This helps to explain why classical WEB did not become a mass standard for ordinary software development.

Yet Donald Knuth's forecast proved more accurate than it might initially appear. He anticipates a future in which the philosophy of WEB is embedded in more efficient programming environments: tangling and compiling might be combined, debugging might operate in terms of the WEB source, and program sections might be displayed on demand rather than requiring the entire document to be printed. This already resembles IDEs, symbol navigation, selective rendering, notebook environments, and contemporary context-management tools.

These lines of development can then be brought together in a two-track conclusion: WEB-like source generation and executable-document narrative.

The final section thus transforms the particular experience of WEB into a broader conclusion: classical WEB remained a niche tool, but its principles spread far more widely.

Conclusion. 1. What Donald Knuth Actually Proposed

Donald Knuth's central proposal cannot be reduced to improved commenting. A comment usually remains a secondary layer: it explains code that has already been organised in an order determined by the compiler, the programming language, or the author's habits. *Literate programming* proposes a stronger model: a program should be constructed from the outset as an explanatory document intended for human reading.

In this model, the central object is not the generated `.pas`, `.c`, `.cpp`, or `.html` file, but a unified source from which different representations are derived:

```
literate source
  → machine-oriented artifact
  → human-oriented document
```

In Donald Knuth's implementation, this scheme is realised through WEB, TeX, Pascal, TANGLE, and WEAVE, but the underlying principle is broader than the original technological pairing. What matters is not the specific historical toolchain, but the discipline: explanation, code, and the mechanism for producing artifacts must remain connected. This is precisely why a program in Donald Knuth's model ceases to be a linear file. It becomes a network of named fragments in which the order of exposition can follow the logic of understanding rather than the sequence required by the machine.

Code is part of the explanatory structure; this remains the principal criterion for evaluating contemporary continuations and reinterpretations of literate programming.

Conclusion. 2. What Persisted and What Changed Form

Classical WEB did not become a mass standard for software development, but several of its principles proved durable. They continued to exist not within a single lineage, but across different engineering and research practices.

The first lineage is **source generation**. CWEB, noweb, and Org Babel preserve the idea closest to WEB: a structured source exists from which machine-oriented code can be produced. Named fragments, tangling, and the prohibition against manually editing derived artifacts are particularly important within this lineage.

The second lineage is **executable documents**. Quarto, Jupyter, and R Markdown shift the emphasis from generating program source code to producing a reproducible document in

which text, code, execution, and results belong to a single process. This is no longer WEB in the literal sense, but the task remains related: a computation must not only be performed, but also explained, rebuilt, and verified.

The third lineage is **navigation within complex systems**. Donald Knuth's section on programs as webs proved particularly farsighted. Contemporary IDEs, dependency graphs, symbol navigation, module maps, knowledge graphs, and context maps address a similar problem: a program is understood not through the linear reading of files, but through a network of relationships between fragments.

The fourth lineage is **the reproducible environment**. In classical WEB, portability was supported through a master source and change files. In contemporary practice, WSL, containers, CI, dev containers, lockfiles, build presets, and documented execution commands partially fulfil the same role. The companion suite for this review was verified within a WSL environment under Windows 11, which illustrates the new form of an old question: what must be reproduced is not only the code, but also the environment in which the code becomes a verifiable artifact.

It is therefore more accurate to speak not of a single successor to WEB, but of a distributed landscape of continuations, reinterpretations, and conceptual associations. `01-cweb`, `02-noweb-like`, and `03-org-babel` form the WEB-like / source-generation lineage. `04-quarto`, `05-jupyter`, and `06-rmarkdown` form the executable-document / computational-publishing lineage. These branches differ technically, but each continues Donald Knuth's central inquiry in its own way: how to connect a program, its explanation, and a verifiable result.

Conclusion. 3. The Cost of the Method and Its Limits of Applicability

Literate programming is not a universal replacement for ordinary software development. Its strength becomes apparent where understanding the program is itself part of the production result: in complex algorithms, research computing, systems with long life cycles, domain-specific rules, generators, DSLs, architectural pipelines, educational materials, and projects in which the cost of an error exceeds the cost of explaining the system in advance.

Its limitations are also substantial. Classical WEB requires proficiency in several languages and modes of work: the document language, the programming language, the syntax of the literate system, and the domain problem itself. Even the lightweight noweb-like approach retains the cost of discipline: the canonical source must be maintained, tangling/rendering must be performed, generated artifacts must not be edited manually, and the result must be checked regularly.

This cost is not a defect of the method; it defines its scope of applicability. For short scripts, one-off utilities, and trivial CRUD code, a full literate process will often be excessive. For systems that must be explained, maintained, verified, and transferred to other people, it becomes an engineering means of reducing the future cost of misunderstanding.

A cautious connection with the language of engineering patterns is appropriate here. Literate programming is not a GoF pattern in the strict catalogue sense (Gamma et al. 1994). It can, however, be understood as a durable pattern for organising source and thought: a recurring solution to a problem in which code, explanation, and verification must remain connected. In this sense, it is closer not to a specific object-oriented template, but to a methodological framework for design.

This concludes the retrospective part of the review proper. The scope will now be deliberately expanded: the discussion will concern not only the forms that literate programming has historically taken and the lineages that can be traced through contemporary tools, but also the ways in which Donald Knuth's central principle may be reinterpreted under the conditions of LLM-assisted development. This is not a claim of direct historical succession, but an authorial development of the problem posed by Knuth.

Conclusion. 4. LLMs: A Continuation of the Idea or a New Form of Rupture

Large language models (LLMs) make Donald Knuth's question newly urgent. If the source of truth is reduced to a prompt alone, literate programming does not emerge. What results is the rapid generation of code from an intention, but not necessarily an explainable, verifiable, and maintainable source. A prompt may be a useful input, but by itself it rarely records the architecture, constraints, invariants, test scenarios, and traceability of the accepted decision.

The difference between TANGLE and an LLM is fundamental. TANGLE deterministically transforms a structured source into a program artifact. An LLM probabilistically continues text on the basis of context, instructions, and examples. LLM output therefore cannot be treated as a build branch: it must be regarded as a candidate artifact that undergoes review and testing before being incorporated back into a managed source.

A strong contemporary version looks as follows:

```
human-authored literate plan
  → chunk contracts
  → bounded prompt
```

```
→ LLM-generated candidate
→ review
→ tests / smoke-check
→ TRACE
→ accepted update to canonical source
```

In this model, the LLM is not the architect by default. It operates within a structure defined by a human: sections, chunks, constraints, expected behaviour, and acceptance criteria must exist before generation, or must be refined before the next run. Otherwise, the LLM does not continue literate programming; it merely accelerates the production of code that must subsequently be understood from scratch.

Adjacent practices — scripting, dialogue systems, rule engines, query parsers, domain-specific languages (DSLs)[12], and behaviour trees — help reveal the shared engineering motive. An intermediate language of intentions often appears between the human and the machine. In a DSL or rule engine, however, this language is usually formalised, whereas an LLM introduces probabilistic interpretation. For AI-assisted development, the discipline of source, contracts, tests, and traceability therefore becomes more important.

This is precisely where the boundary with vibe coding lies. A simple prompt → code scheme may be productive for drafting, but it does not in itself continue literate programming. A substantive continuation is possible only within a workflow in which the prompt is subordinate to an explanatory source, while the result is verified and returned to a traceable project structure.

This is why the companion suite contains no deterministic `07-llm` equivalent to CWEB, noweb, Org Babel, Quarto, Jupyter, or R Markdown. Instead, `07-prompt-literate` has been introduced as a demonstrative application of **Prompt-Literate Workflow**. Its purpose is not to reproduce LLM output bit for bit, but to demonstrate a controlled process: a human-authored plan, chunk contracts, bounded prompts, review, smoke-check, and TRACE.

`07-prompt-literate` does not redefine Prompt-Literate Workflow. It is a local example of applying the method within the article to Donald Knuth's prime-number problem. Result-specific markers and local checks are implemented as an additive extension of the core methodology: it introduces local verification rules without overriding the basic invariants.

[12] A domain-specific language, or DSL, is a language deliberately restricted to the tasks of a particular domain.

Industrial tools are already moving in this direction: VS Code / GitHub Copilot custom instructions, prompt files, agent customisation, skills/hooks, and MCP-like mechanisms[13] establish a persistent project context ("Customize GitHub Copilot in Visual Studio Code," n.d.; "Customizing GitHub Copilot," n.d.). Regulatory requirements and provenance standards[14] provide an additional context: the EU AI Act[15] and the General-Purpose AI Code of Practice[16] intensify the demand for transparency and accountability, while C2PA[17] provides a technical model for recording the provenance and modification history of digital content. These remain indicators of transitional standardisation, however, rather than conclusive evidence that AI-assisted coding has already become literate programming.

Conclusion. 5. Where the Idea May Develop Further

The most evident area of application for the literate approach today is scientific and analytical documentation. In this domain, the connection between text, code, and computation — a connection closely related to Donald Knuth's original formulation — has already become an everyday practice: R Markdown, Quarto, knitr, Org Babel, and notebook environments connect text, code, data, computations, graphs, and publication. In these fields, explanation is not an embellishment; it is necessary for verifying the result.

The second area is education. The literate approach is particularly useful where it is necessary to show not only the final code, but also the reasoning behind it: algorithms, data structures, systems programming, C++, and the analysis of legacy code. A program becomes not a "correct answer" appended to the end of educational material, but a route towards understanding.

The third area is architectural and tooling documentation: build pipelines, code generation, DSLs, configuration systems, runtime contracts, test scenarios, and reproducible environments. Conventional documentation placed alongside the code quickly becomes outdated if it is not connected to artifacts and verification procedures. A literate source does not solve this problem automatically, but it compels the author to define the source of truth explicitly.

[13] Model Context Protocol, or MCP, is an open protocol for connecting AI applications to external systems, data sources, and tools.
[14] Provenance denotes information about an artifact's origin, authorship, and chain of modifications.
[15] The EU AI Act is the European Union regulation that sets risk-based requirements for artificial-intelligence systems.
[16] The General-Purpose AI Code of Practice is a voluntary EU framework designed to help providers of general-purpose AI models comply with the requirements of the EU AI Act.
[17] C2PA is a technical specification developed by the Coalition for Content Provenance and Authenticity for machine-readable records of digital-content origin and modification history.

The fourth area is game development and game design document (GDD) pipelines[18]. Here, Donald Knuth's idea can be extended beyond programming in the strict sense. A GDD already occupies an intermediate position between literary description, specification, a system of rules, and future implementation. A good GDD does not merely describe a game: it should generate scenes, mechanics, data, quest structures, test scenarios, tasks, and verifiable artifacts.

At this point, a broader framework emerges: **literate design**.

```
GDD source
  → narrative documentation
  → mechanics/spec chunks
  → quest structures
  → data schemas
  → implementation tasks
  → validation checks
  → generated wiki / build artifacts
```

This is not a literal continuation of WEB, but it extends the original question into another domain: how can a complex system be described in a form from which working representations and verifiable artifacts can be produced?

For our projects, this review has a practical purpose. It did not originate as a nostalgic commentary on an article published in 1984, but as preparation for a pipeline in which a unified explanatory document can connect a GDD, code, tests, a wiki, generated artifacts, and LLM-assisted work. Our source must be sufficiently expressive to explain the system and sufficiently disciplined to generate verifiable results.

Practical application has shown that a universal workflow must remain compact. Domain-specific rules — the architecture of a particular project, runtime integration, persistence, authoring tools, and the publication pipeline — should be introduced as project-local extensions and must under no circumstances become part of the core method.

Literate programming is not required everywhere. But where understanding is itself part of production, its central idea remains relevant: first construct an explanatory source, then derive machine-oriented and human-oriented representations from it, rather than attempting to reconstruct meaning after the code has already emerged.

[18] A game design document, or GDD, is a project document that connects a game's concept, rules, content, systems, and future implementation.

A. Primary Knuth / WEB / CWEB, structured-programming, and software-design context

Dijkstra, E.W. 1972: 'Notes on Structured Programming'. In O.-J. Dahl et al. (eds), *Structured Programming*. Academic Press, 1–82.

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. 1994: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Knuth, D.E. 1984: 'Literate Programming'. *The Computer Journal*. DOI: 10.1093/comjnl/27.2.97.

Knuth, D.E. and Levy, S. n.d. 'The CWEB System of Structured Documentation'. https://www-cs-faculty.stanford.edu/~knuth/cweb.html.

Martin, R.C. 2017: *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, 1st edn. Prentice Hall.

B. Direct literate-programming tools and descendants

'GNU Emacs Download' n.d. https://www.gnu.org/software/emacs/download.html.

'Org Manual: Extracting Source Code' n.d. https://orgmode.org/manual/Extracting-Source-Code.html.

Ramsey, N. n.d. 'Noweb Repository'. https://github.com/nrnrnr/noweb.

C. Executable documents / reproducible research / notebook lineage

'Knitr' n.d. https://yihui.org/knitr/.

'Nbconvert Documentation' n.d. https://nbconvert.readthedocs.io/.

'Pandoc' n.d. https://pandoc.org/.

'Project Jupyter Documentation' n.d. https://docs.jupyter.org/.

'Quarto Documentation' n.d. https://quarto.org/docs/.

'R Markdown Documentation' n.d. https://rmarkdown.rstudio.com/.

D. LLM / AI-assisted programming

‘Customize GitHub Copilot in Visual Studio Code’ n.d. https://code.visualstudio.com/docs/copilot/copilot-customization.

‘Customizing GitHub Copilot’ n.d. https://docs.github.com/en/copilot/how-tos/custom-instructions.

E. Governance / provenance (contextual)

‘C2PA Specification’ n.d. https://c2pa.org/specifications/.

‘General-Purpose AI Code of Practice’ n.d. https://digital-strategy.ec.europa.eu/en/policies/ai-code-practice.

Regulation (EU) 2024/1689 (Artificial Intelligence Act) n.d. https://eur-lex.europa.eu/eli/reg/2024/1689/oj.